Documento 03 · Cuaderno de laboratorio

Para: alumnos · DeepSeek Harness (dsh) · Generado el 03/11/2026

Cuaderno de laboratorio — Del chat al agente

Seminario: Del chat al agente: qué es un harness y cómo se usa. **Qué es esto:** tu cuaderno de prácticas. Lo vas a usar con el portátil delante, escribiendo de verdad. **Cuánto dura:** unas dos horas y media repartidas en seis prácticas. La sexta es de análisis de datos con Python.

Regla de la sala: si te atascas más de tres minutos, levanta la mano. No hay nada que haga perder más tiempo que diez personas atascadas en silencio.

Antes de empezar: la carpeta de práctica

Vas a trabajar **siempre dentro de una copia de la carpeta de práctica**. Eso no es un capricho: es tu principal medida de seguridad. Usaremos un expediente ficticio de una empresa de ejemplo, preparado para el curso.

1. **Descarga `descarga_entorno.zip`** (también está en la página Entorno de práctica del curso) y descomprímelo en un lugar fácil de encontrar, como Documentos.
2. Cambia el nombre de la carpeta extraída a `practica_agente`.
3. Conserva aparte el ZIP original como copia limpia para empezar de nuevo si hace falta.
4. Comprueba que dentro de `practica_agente` hay una carpeta `informes`, tres archivos CSV con su `diccionario_datos.md` (qué significa cada columna y en qué unidad está), los documentos de configuración y reunión, y los archivos para arrancar opencode (`1_instalar_opencode` y `2_arrancar_opencode`).
5. **Trabaja siempre en la copia `practica_agente`**, nunca en el ZIP ni en otra carpeta con tus archivos. El agente puede modificar lo que haya dentro.

Atención: **Por qué insistimos con la copia de seguridad:** el agente va a poder leer y modificar los archivos de esta carpeta, y va a ejecutar acciones de verdad. No es una simulación. Si prefieres practicar con algo que no te importe perder, mejor.

Qué necesitas tener a mano:

- opencode y Python instalados (lo hiciste en la preparación previa). No necesitas clave de API: usaremos sus modelos gratuitos.
- El portátil con el cargador enchufado.

Si te atascas más de tres minutos, no sigas peleándote en silencio: mira la tabla **«Si algo falla»** de la práctica y, si estás en clase, levanta la mano.

Práctica 1 — Primer arranque y primer encargo

Tiempo: unos 20 minutos. **Objetivo:** tener el harness funcionando y haber completado un encargo de principio a fin.

Paso 1 — Arranca opencode con doble clic

Abre la carpeta `practica_agente` y haz doble clic en **`2_arrancar_opencode.bat`** (en macOS, `2_arrancar_opencode.command`). No hace falta escribir nada.

Se abre una **ventana negra** que muestra la carpeta de trabajo y la versión de opencode, y después se abre **sola** una página en tu navegador con la interfaz de opencode, en español. Si no se abre, copia la dirección que aparece en la ventana (algo como `http://127.0.0.1:4096`) y pégala en el navegador.

- **Si la versión es `1.19.5`**, todo bien. Es la versión probada en este curso.
- **Si es otro número**, cierra la ventana y haz doble clic en `1_instalar_opencode`: instala la del curso para que las prácticas de permisos funcionen como se describen.
- **Si dice [FALTA] opencode**, vuelve a la preparación previa: falta Node.js o falta ejecutar el instalador.

No cierres la ventana negra. Si la cierras, se apaga el agente. Cerrarla y volver a hacer doble clic es también la forma de **reiniciarlo**.

¿Con terminal? Es lo mismo: `opencode --version` para ver la versión y, dentro de la carpeta (`cd Documents\practica_agente`; en macOS, `cd ~/Documents/practica_agente`), `opencode web` para arrancar. Para apagarlo, **Ctrl + C**.

Paso 2 — Abre tu carpeta como proyecto

1. Si no ves tu carpeta en la lista de **Proyectos**, pulsa **Añadir proyecto**.
2. En el buscador, escribe `Documents/practica_agente` y elige tu carpeta. **Cuidado:** un clic sobre una carpeta puede añadirla directamente como proyecto. Comprueba que la que aparece en **Proyectos** es exactamente `practica_agente` y no una carpeta más amplia como `Documents`. Si te equivocas, cierra ese proyecto y vuelve a empezar: una carpeta matriz como sala da al agente acceso a todo lo que contiene.
3. Pulsa **Nueva sesión**.

Esa carpeta es la **sala** del agente: lo que hay dentro lo puede tocar; para salir de ella te pedirá permiso. Ojo: la sala es una **regla de permisos, no un muro**. opencode no aísla el programa del resto del ordenador (no tiene *sandbox*); lo verás en la Práctica 3.

Paso 3 — Comprueba el modelo

Debajo del cuadro de mensajes verás el nombre del modelo, por ejemplo **Big Pickle**. Es uno de los **modelos gratuitos** de opencode: no necesitas cuenta ni clave de API.

Pulsa sobre el nombre: se abre **Seleccionar modelo**, con los gratuitos marcados como **Gratis**. Elige **GLM-5.3-Flash** (o **DeepSeek V4.1 Flash**). También son gratuitos y, según la página de privacidad de opencode, sus proveedores no guardan tus datos ni los usan para entrenar; con Big Pickle, en cambio, los datos pueden usarse para mejorar el modelo. *(Datos a 30 de octubre de 2026: los modelos gratuitos cambian a menudo. Si no aparecen, elige otro marcado como gratuito y mira su política en la documentación de opencode.)*

Gratis no quiere decir privado. **No pongas en esta carpeta datos reales ni confidenciales**: lo que el agente lee se envía al proveedor del modelo.

Si usas Claude Code o Codex (carril B): en vez de los pasos 1 a 3, usa la aplicación de escritorio, sin terminal. **Claude:** abre la app, ve a la pestaña **Code**, elige **Local** y selecciona la carpeta `practica_agente` (en Windows, la primera vez te pedirá instalar **Git for Windows**). **Codex:** abre la aplicación Codex y abre la carpeta `practica_agente`. La primera vez te pedirá iniciar sesión con tu cuenta y confirmar que confías en la carpeta. El modelo ya viene incluido en tu plan; no hay que elegir uno gratuito. Los datos que envías siguen la política de tu cuenta: revísala igual. *(Si prefieres la terminal: `claude` o `codex` dentro de la carpeta.)*

Paso 4 — Tu primer encargo

Escribe esto tal cual, cambiando lo que haga falta para que encaje con tu carpeta:

Mira qué hay en la carpeta `informes` y hazme un índice de los archivos, agrupados por tipo, en un archivo nuevo llamado `INDICE.md`, guardado en la raíz de la carpeta de trabajo (no dentro de `informes`). No modifiques ningún archivo original.

Qué deberías ver

- Que el agente **no responde de golpe**: va dando pasos (*Explorado*, *Shell*, *Escribir*…).

- Que cada paso aparece en pantalla, indicando qué está haciendo.
- Que al final te dice **dónde ha dejado** el archivo nuevo. Compruébalo tú: busca `INDICE.md` en la raíz de `practica_agente`, junto a la carpeta `informes`. Si un encargo no dice dónde guardar el resultado, el agente elige, y puede elegir otro sitio.

Hecho Comprobación

- [ ] Hay un archivo nuevo, `INDICE.md`, en la raíz de tu carpeta, con ese nombre exacto.
- [ ] Lo has abierto y el contenido tiene sentido.
- [ ] Los archivos originales siguen ahí, intactos.
- [ ] Sabrías decir **qué herramientas** ha usado el agente para hacerlo (mirar la carpeta, leer, ejecutar una orden, escribir...).

Si algo falla

Lo que ves	Qué hacer
La ventana negra dice [FALTA] opencode o se cierra al momento	Haz doble clic en 1_instalar_opencode y lee qué línea no dice [OK]. Si sigue, repite la preparación previa.
La página no carga	Comprueba que la ventana negra sigue abierta y copia exactamente la dirección que aparece en ella.
No encuentras tu carpeta en Añadir proyecto	Escribe la ruta desde tu usuario, por ejemplo Documents/practica_agente. Las carpetas ocultas o temporales no salen.
El modelo responde con un error o dice que está saturado	Los gratuitos a veces se saturan. Pulsa el nombre del modelo, elige otro marcado como Gratis y repite.
El agente tarda mucho	Está trabajando. Mira los pasos: si avanzan, déjalo.

Práctica 2 — Provocar una aprobación

Tiempo: unos 10 minutos. **Objetivo:** ver con tus ojos qué pasa cuando el agente intenta salir de su zona, y entender qué te está pidiendo exactamente.

Esta es la práctica más importante del curso. Léela entera antes de empezar.

Paso 1 — Pídele algo que está fuera de su zona

Primero, **tú a mano**, crea en el escritorio una carpeta vacía llamada `prueba_permisos`. Será la zona de pruebas de fuera: desechable y sin nada que importe.

Asegúrate de que estás en una sesión del proyecto `practica_agente`. Ahora pídele algo que ocurra **fuera** de ella, en esa carpeta:

Crea un archivo llamado `prueba_agente.txt` en la carpeta `prueba_permisos` del escritorio de mi usuario, con el texto «prueba».

Paso 2 — Para y mira

No apruebes nada todavía. Va a aparecer un aviso **«Permiso requerido»**: *Acceder a archivos fuera del directorio del proyecto*, con una ruta debajo. Lee el aviso entero, despacio, antes de tocar nada.

Lee la ruta que cubre el permiso. Puede no ser solo el archivo que pediste, sino **toda una carpeta**: por

ejemplo `...\Desktop\prueba_permisos\*`, donde el `*` significa «todo lo que hay dentro». Si el alcance es más amplio de lo que esperabas (por ejemplo, todo el escritorio o tu carpeta de documentos), pulsa **Denegar** y vuelve a pedirlo de forma más acotada.

Tiene tres botones: **Denegar**, **Permitir siempre** y **Permitir una vez**.

Atención: **Otros agentes no siempre preguntan.** En algunos, como Claude Code en *modo Auto*, otra IA decide por ti si la acción es arriesgada y un archivo inofensivo en el escritorio suele pasar sin aviso. Sin aviso, el freno de «leer antes de aprobar» desaparece. Cuando uses otra herramienta, comprueba primero cómo pregunta.

Si usas Claude Code o Codex (carril B): el aviso tiene otro aspecto y otras palabras, pero las opciones son equivalentes: sí esta vez, sí a partir de ahora, o no. En Claude Code, trabaja en el modo normal (el de serie), no en modo Auto. En Codex, deja el modo de serie, que permite trabajar dentro de la carpeta y pide permiso para salir de ella. Las tres preguntas del paso 3 son exactamente las mismas.

Paso 3 — Hazte las tres preguntas

Antes de decidir, contesta por escrito:

¿Qué me está pidiendo exactamente? (¿leer un archivo? ¿escribir uno nuevo? ¿ejecutar una orden? ¿dónde? ¿el permiso cubre solo ese archivo o una carpeta entera?)

¿Qué pasaría si digo que sí? (Piensa en algo concreto: qué archivo aparecería, dónde, y qué cambiaría en tu ordenador.)

¿Puedo acotarlo? (¿Podrías pedirle que lo haga dentro de la carpeta de trabajo en vez de en el escritorio?)

Paso 4 — Decide, conscientemente

Ahora sí: pulsa **Permitir una vez**. Y observa lo que pasa.

Paso 5 — El detalle que hay que entender

Repite la petición otra vez:

Crea un archivo llamado `prueba_agente2.txt` en la carpeta `prueba_permisos` del escritorio.

Te va a volver a preguntar. Esta vez pulsa **Denegar** y mira qué hace: el agente se para y no lo intenta por otro camino. Denegar también es un buen resultado: has comprobado que el freno funciona. Si después de denegar el agente buscara otra forma de hacerlo (otra ruta, otra orden), eso sí sería un problema: apúntalo y avisa al instructor.

«Permitir una vez» es permitir esa vez. No vale para siempre.

Es incómodo **a propósito**: ese pequeño fastidio es lo que te obliga a pensar si de verdad quieres que el agente salga de su zona. Un permiso que se diera «para siempre» con un clic sería más cómodo y mucho más peligroso.

Paso 6 — Limpia

Borra el archivo de prueba, `prueba_agente.txt`. Deja la carpeta `prueba_permisos`: la usarás en la Práctica 3.

Hecho Comprobación

- [ ] He visto aparecer un aviso de aprobación.

- [ ] He leído el aviso **antes** de aprobarlo, incluida la ruta que cubre.
- [ ] Sé explicar con mis palabras qué me estaba pidiendo.
- [ ] He comprobado que al repetir la petición **vuelve a preguntar**, y he visto qué pasa al denegar.
- [ ] He borrado los archivos de prueba.

Los dos malos hábitos que ya puedes reconocer

- **Aprobar por inercia:** después del aviso número veinte, pulsas «permitir» sin leer. En ese momento el aviso ha dejado de protegerte.
- **Subir el nivel de permiso para que deje de preguntar:** es como quitar el freno porque el coche hace ruido al frenar.

Regla de oro: si no entiendes qué te está pidiendo, **no lo apruebes**. Pídele que te lo explique con palabras o que te diga qué va a tocar antes de tocarlo.

Práctica 3 — Cambiar de política

Tiempo: unos 8 minutos. **Objetivo:** comprobar que dentro de la sala no pregunta, y que **tú puedes cambiar la política** para que pregunte también ahí.

Paso 1 — La misma tarea, pero dentro

Pide lo mismo que antes, esta vez **dentro** de tu carpeta de trabajo:

Crea un archivo llamado `prueba_dentro.txt` en la carpeta de trabajo, con el texto «prueba».

Paso 2 — Observa

No ha preguntado nada. Lo ha hecho y punto. Es la política que trae opencode de serie: **dentro de la carpeta puede leer, escribir y ejecutar órdenes sin preguntar**; solo pide permiso para salir.

Paso 3 — Cambia la política

La política se guarda en un archivo, `opencode.json`, en la raíz de tu carpeta. Pídeselo al propio agente:

Crea en la raíz de esta carpeta un archivo `opencode.json` con exactamente este contenido: `{"$schema": "https://opencode.ai/config.json", "permission": {"edit": "ask", "bash": "ask", "external_directory": "ask"}}`

Lo que dice: antes de **editar** cualquier archivo (`edit`) o **ejecutar** cualquier orden (`bash`), pregunta (`ask`); y para cualquier ruta **fuera de la carpeta** (`external_directory`), pregunta también. Esta última ya viene así de serie en la versión 1.19.5, pero la escribimos para que tu protección no dependa de la versión. Referencia: permisos de opencode.

Paso 4 — Reinicia opencode

La política se lee al arrancar. Si pruebas ahora, seguirá sin preguntar. Así que:

1. Cierra la ventana negra de opencode para apagarlo.
2. Vuelve a arrancarlo con doble clic en `2_arrancar_opencode`.
3. Abre una **Nueva sesión** en tu proyecto.

Paso 5 — Repite la prueba

Crea un archivo llamado `prueba_dentro2.txt` en la carpeta de trabajo, con el texto «prueba».

Ahora sí pregunta, aunque sea dentro de la sala: *Modificar archivos mediante ediciones, escrituras y parches*. Pulsa **Permitir una vez**.

Comprueba también que la puerta sigue cerrada:

Crea un archivo llamado `prueba_fuera.txt` en la carpeta `prueba_permisos` del escritorio de mi usuario.

Tiene que salir el aviso *Acceder a archivos fuera del directorio del proyecto*. Pulsa **Denegar**.

Si no pregunta en alguno de los dos casos, para: no sigas creyendo que tienes una protección que no has visto funcionar. Comprueba que reiniciaste opencode y que la ventana negra muestra la versión 1.19.5. Si la interfaz o los avisos no se parecen a lo que describe este cuaderno, avisa al instructor.

Paso 6 — Saca la conclusión

El agente no es más listo ni más tonto. Has cambiado sus reglas.

Hay dos palancas, y las dos son tuyas:

- **Dónde trabaja** (la carpeta): cuanto más pequeña, menos riesgo.
- **Qué puede hacer sin preguntar** (la política): más avisos, más control; menos avisos, más velocidad.

Si una tarea te obliga a aprobar cien avisos, el problema no es el agente: es dónde está trabajando o qué política le has puesto. Mueve el trabajo dentro de la carpeta y ajusta la política; no apruebes a ciegas.

Paso 7 — Déjalo como estaba

Borra la carpeta `prueba_permisos` del escritorio. Para las prácticas siguientes, **borra `opencode.json`** y reinicia opencode (cierra la ventana negra de opencode y vuelve a hacer doble clic en `2_arrancar_opencode`). Si prefieres seguir aprobando cada cambio, déjalo: solo tendrás más avisos.

Si usas Claude Code o Codex (carril B): la prueba es la misma, pero la política de serie es distinta, así que puede que los resultados sean los contrarios.

- **Claude Code** pregunta de serie antes de editar un archivo, también dentro de la carpeta. Pulsa **Mayús + Tab** hasta ver *accept edits* (aceptar ediciones) y repite: verás que deja de preguntar. Vuelve al modo normal con Mayús + Tab. La política fija se guarda en `.claude/settings.json`.
- **Codex**: escribe `/permissions` y elige un modo más estricto (por ejemplo, solo lectura). Repite la prueba y vuelve después al modo de serie.

La conclusión no cambia: **las reglas las pones tú**.

Hecho Comprobación

- [ ] El archivo `prueba_dentro.txt` se creó **sin** pedir aprobación.
- [ ] Con `opencode.json` y tras reiniciar, `prueba_dentro2.txt` **sí** pidió aprobación.
- [ ] Escribir en `prueba_permisos` pidió aprobación y lo denegué.
- [ ] Sé explicar la diferencia entre la carpeta y la política en una frase.

Y no lo olvides

La carpeta reduce el riesgo, no lo elimina. Con la política de serie, todo lo que está **dentro** de la carpeta el agente lo puede tocar sin preguntar, incluidas las órdenes de terminal. opencode detecta cuándo una orden apunta fuera de la carpeta y te pregunta, pero **no es un sandbox**: no aísla el programa del resto del

ordenador. Si dentro hay algo que no debería tocarse, el problema es lo que hay en la carpeta.

Práctica 4 — Enseñarle un procedimiento

Tiempo: unos 5 minutos (más si te animas). **Objetivo:** dejar una instrucción permanente y comprobar que sobrevive a cerrar la conversación.

La idea

Hasta ahora, cada vez que quieres algo tienes que explicárselo. Si tienes una preferencia que se repite, **no la repitas cada día: escríbela una vez.**

`AGENTS.md` son las notas que dejas al becario antes de que empiece.

Paso 1 — Pide que cree sus propias notas

Lo más fácil es pedírselo al propio agente:

Crea en la raíz de esta carpeta de trabajo un archivo `AGENTS.md` con estas instrucciones para ti:

- Responde siempre en español.
- Si creas un archivo de informe, guárdalo en la carpeta `informes`. Si solo respondes en el chat, no crees archivos.
- Nunca modifiques ni borres los archivos originales: si necesitas cambiarlos, crea una copia.
- Al terminar cualquier tarea, dime qué archivos has tocado.

(Si prefieres escribirlo tú a mano, crea el archivo `AGENTS.md` en la raíz de la carpeta de trabajo y pega esas mismas líneas.)

Antes de seguir, compruébalo tú. Abre la carpeta `practica_agente` en el explorador de archivos: `AGENTS.md` tiene que estar en la raíz, junto a `informes`. Ábrelo y comprueba que contiene las cuatro reglas.

Si el agente dice que lo ha creado y no está, no es raro: a veces un agente da por hecho algo que no ha llegado a hacer. Pídeselo otra vez exigiendo la prueba: «Crea `AGENTS.md` en la raíz, comprueba que existe y enséñame su contenido». Y vuelve a mirarlo tú. Si sigue sin aparecer, créalo a mano.

Paso 2 — El detalle que hace que esto sea una prueba de verdad

Abre una conversación nueva. No sigas en la misma.

En la misma conversación no demuestra nada: el agente se acuerda de lo que le has dicho porque lo tiene delante, en su mesa de trabajo. **La prueba de verdad es empezar de cero.**

Paso 3 — Comprueba que las respeta

En la conversación nueva, pídele algo normal, por ejemplo:

Hazme un resumen de los archivos de la carpeta `informes`.

Y observa si cumple las notas: ¿responde en español?, ¿guarda el resultado en `informes`?, ¿te dice qué archivos ha tocado?

Hecho Comprobación

- [ ] He visto con mis ojos `AGENTS.md` en la raíz de mi carpeta de trabajo, con las cuatro reglas, **antes** de abrir la conversación nueva.
- [ ] He abierto una **conversación nueva**.
- [ ] El agente ha respetado las instrucciones **sin que se las repita**.
- [ ] Sé explicar la diferencia entre lo que dura una conversación y lo que es permanente.

Si te sobra tiempo: un procedimiento guardado (skill)

Las notas son preferencias. Los **procedimientos guardados** (skills) son para tareas que repites siempre igual:

Las skills son procedimientos guardados, como recetas de cocina.

Pídele al agente que te cree una:

Crea una skill llamada `resumen-informes` que consista en: leer los informes `.txt` de la carpeta `informes` (no el propio resumen ni otros archivos), crear o sustituir `informes/resumen_general.md` con una tabla de tres columnas (archivo, tema, conclusión) y no modificar los originales.

Fíjate en que la ruta de salida, `informes/resumen_general.md`, es la misma que dicen tus notas de `AGENTS.md`. **Las notas y la skill no deben competir:** si una dice una carpeta y la otra otra, no sabrás cuál seguirá el agente.

opencode la guarda en `.opencode/skills/resumen-informes/SKILL.md`. **Las skills se cargan al arrancar**: cierra la ventana negra de opencode y vuelve a hacer doble clic en `2_arrancar_opencode` y, en una sesión nueva, pídele «hazme el resumen de informes». Comprueba que sigue la receta: debe crear `informes/resumen_general.md` (no en la raíz) con una fila por informe y sin una fila para el propio resumen. Si no reinicias, responderá en el chat sin usarla.

Si usas Claude Code o Codex (carril B):

- **Codex** lee `AGENTS.md` directamente. Guarda las skills en `.agents/skills/<nombre>/SKILL.md`.
- **Claude Code** lee `CLAUDE.md`, no `AGENTS.md`. Pídele: «Crea un `CLAUDE.md` que contenga solo la línea `@AGENTS.md`». Así los dos agentes comparten las mismas notas. Guarda las skills en `.claude/skills/<nombre>/SKILL.md`.
- En los dos casos, si no reconoce la skill, abre una sesión nueva. El archivo `SKILL.md` es el mismo formato: una skill escrita en opencode te sirve copiándola a la carpeta correspondiente.

La lección, para llevártela

Si te descubres explicando lo mismo por tercera vez, el problema no es el agente: es que no le has dejado notas.

Práctica 5 — Encargo integrador

Tiempo: unos 13 minutos. **Objetivo:** hacer un encargo completo tú solo, y **verificarlo**. Aquí ya no hay instrucciones que copiar.

Paso 1 — Piensa una tarea real tuya

Pequeña, que tenga que ver con archivos, y que puedas comprobar en dos minutos. Por ejemplo: ordenar archivos por tipo, sacar un listado, resumir varios documentos, extraer datos repetidos, unificar notas sueltas.

Paso 2 — Redacta el encargo con las cuatro partes

Este patrón es lo más útil que te llevas hoy. Cópialo de aquí en adelante:

1. Qué quiero — el objetivo, con un resultado concreto. **2. Dónde** — la carpeta exacta. **3. Qué NO tocar** —

los límites, dichos explícitamente. **4. Cómo sabré que está bien** — tu criterio de éxito.

Ejemplo completo:

«En la carpeta `practica_agente/informes` hay cinco informes en texto (`informe_*.txt`). Léelos y créame `informes/resumen_general.md` con una tabla de tres columnas: tema, conclusión y nivel de urgencia. **No modifiques ninguno de los cinco archivos originales.** Al terminar, dime qué archivos has leído y cuántas filas tiene la tabla.»

Mi encargo:

```
1. Qué quiero: _____________________________________________
2. Dónde: ___________________________________________________
3. Qué NO tocar: ____________________________________________
4. Cómo lo comprobaré: ______________________________________
```

Paso 3 — Lánzalo y mira los pasos

No te vayas a por un café. **Mira cómo trabaja.** Cada paso te dice qué herramienta está usando. Ahí es donde te darás cuenta si va por donde no debe.

Paso 4 — Verifica. Esta es la parte importante

Cuando diga «hecho», **no está hecho: está propuesto.**

El agente no firma su trabajo. Lo firmas tú cuando dices que está bien.

- [ ] He **abierto** el archivo resultante (no vale «tiene buena pinta»).
- [ ] He **contado**: ¿están todos los elementos?, ¿las filas que debería?
- [ ] Si la tabla selecciona filas por una condición (por ejemplo, «existencias por debajo del mínimo»), he **contrastado cada fila con el archivo original**: todas la cumplen de verdad (existencias < mínimo) y no falta ninguna que la cumpla. Una fila que no cumple la condición es un error, aunque el informe lo presente con seguridad.
- [ ] He comprobado que **los archivos que no debía tocar siguen intactos**.
- [ ] Le he preguntado **qué archivos ha tocado**, y la respuesta cuadra con lo que veo.

Hecho Comprobación

- [ ] He escrito un encargo con las cuatro partes.
- [ ] El agente ha producido un resultado.
- [ ] He verificado el resultado abriéndolo, contando y comprobando las cifras de cada fila.
- [ ] He comprobado que lo que no debía tocarse no se ha tocado.
- [ ] Sé decir **una cosa que revisaría mejor la próxima vez**.

Atención: **Cuidado con esta trampa:** el resultado llega **bien escrito, bien maquetado y con tono seguro**. Eso no significa que sea correcto. Un agente puede equivocarse con una seguridad perfecta.

Práctica 6 — Python: comprobar y modelar datos

Tiempo: unos 25 minutos (si en clase no da tiempo, termina la parte B en casa). **Objetivo:** que el agente **escriba y ejecute programas en Python** sobre tus datos, y aprender a **revisar el programa, no solo el número**.

Hasta ahora, el agente leía y escribía textos. Con datos, lo que de verdad cambia las cosas es que **escribe un programa, lo ejecuta y te da el resultado**. El cálculo queda escrito: lo puedes revisar, repetir con los datos del mes que viene y enseñárselo a otra persona. Un chat que «lee» tu tabla no te deja nada de eso.

Usamos **Python**, el lenguaje más extendido para análisis de datos e ingeniería. Lo instalaste en la preparación

previa, junto con `pandas` (tablas) y `matplotlib` (gráficos). No necesitas saber programar: necesitas saber **qué buscar** en el programa.

Parte A — Comprobar con un programa (10 minutos)

En la Práctica 5 comprobaste a ojo las filas de una tabla. Ahora haz que lo compruebe un programa.

Paso 1 — El encargo. Copia esto tal cual:

«Escribe un script en Python, `analisis/bajo_minimo.py`, que lea `inventario_material.csv` y `contactos_proveedores.csv`, liste los artículos con **existencias por debajo del stock mínimo** y, para cada uno, el nombre del proveedor y su plazo de suministro. Ejecútalo y enséñame la salida tal cual. **No modifiques los CSV.**»

Paso 2 — Mira el programa. Ábrelo (está en `analisis/`). Busca la línea que decide si un artículo está bajo mínimo. Tiene que comparar **existencias < mínimo**, no otra cosa (por ejemplo, «cerca del mínimo»).

Paso 3 — Contrasta con los datos. Abre `inventario_material.csv` y comprueba **tú** dos filas: una que salga en la lista y otra que no salga. ¿Cuadran?

Paso 4 — Repítelo. Pídele «ejecuta otra vez el script» o cambia a mano una cifra en una **copia** del CSV y pídele que lo ejecute sobre la copia. El mismo programa da siempre la misma respuesta: esa es la diferencia con preguntarle otra vez al chat.

Paso 5 — Compara con la solución (solo después de haberlo contrastado tú). Con los datos originales, la lista correcta tiene **dos** artículos: **J-18** (420 frente a un mínimo de 600; proveedor V-104, Elastómeros Técnicos, 8 días laborables) y **S-04** (18 frente a 40; V-101, Instrumentación Bruma, 15 días). Si sale otro, o falta uno, mira en el script la comparación y el cruce entre los dos archivos.

Parte B — Datos y modelo (15 minutos)

En la carpeta está `lecturas_energia_diarias.csv`: el consumo eléctrico diario de la planta ficticia durante las cuatro semanas del informe `informe_04_energia_mayo.txt`, con las unidades producidas y las horas de operación. El informe dice que la semana 3 consumió más y sospecha del compresor por la noche. Vamos a comprobarlo.

Paso 1 — El encargo. Copia este encargo tal cual (tiene las cuatro partes):

«En `practica_agente` está `lecturas_energia_diarias.csv`. Quiero:

1. Antes de calcular nada, dime cuántas filas hay, qué columnas, en qué unidades crees que está cada una y si ves valores vacíos o raros.
2. Escribe un script en Python, `analisis/energia.py`, que calcule el consumo total y por semana, y que ajuste un modelo lineal sencillo: kWh diarios en función de las unidades terminadas.
3. Usa el modelo para decir qué semana consume más de lo que le corresponde por su producción, y guarda un gráfico en `analisis/`. **No modifiques el CSV original.** Si corriges algún dato, hazlo en el script y dime cuál y por qué. Al terminar, dime el consumo total y compáralo con los 32.400 kWh del informe de energía.»

Tres controles para toda la práctica

1. Confirma filas, columnas, unidades y valores vacíos **antes** de calcular.
2. Calcula el total **con y sin** la lectura sospechosa, y no edites el CSV original.
3. Compara el exceso con el modelo ajustado con todos los datos y con la semana 3 fuera del ajuste. Trata los dos resultados como **señales**, no como prueba de la causa.

Si el agente quiere instalar algo (otra librería, otra versión de Python), te pedirá permiso. Piénsalo antes de aceptar: con `pandas` y `matplotlib` basta, y también se puede hacer sin ellas.

Paso 2 — Mira antes de que calcule. Fíjate en la respuesta al punto 1. Hay **dos trampas** puestas a propósito en los datos. ¿Las ha encontrado el agente antes de calcular? Si no, pregúntale: «¿Todas las lecturas de `kwh_total` están en las mismas unidades?».

Paso 3 — Revisa el programa, no solo el número.

- Abre el script. No hace falta entenderlo todo: busca **dónde lee el archivo**, **qué filas descarta o corrige** y **qué fórmula ajusta**.
- **Obligatorio:** pregúntale «¿Has usado los mismos datos para ajustar el modelo y para evaluarlo?» y pídele una segunda estimación: «Ajusta el modelo sin la semana 3 y calcula otra vez su exceso». Si la semana rara entra en el ajuste, tira de la recta hacia ella y su exceso parece menor. Apunta los dos números.

- Pregúntale: «¿Para qué rango de producción vale este modelo?».
- Abre el gráfico que ha guardado en `analisis/`. ¿Se ve la semana que se sale?

Paso 4 — Totales de control. Comprueba **tú** dos números contra el informe original:

- [ ] El consumo total sale **32.400 kWh** si la lectura `0.751` se interpreta como 0,751 MWh (751 kWh), y unos 31.650 sin corregirla. Pide los dos escenarios. Que cuadre con el informe hace muy probable la conversión, pero sigue siendo un **supuesto**: sin una fuente que confirme las unidades, preséntalo así y anota la justificación.
- [ ] Las unidades terminadas suman **17.820**.

Hecho Comprobación

- [ ] **Parte A:** el script `analisis/bajo_minimo.py` existe, compara existencias < mínimo y he contrastado dos filas con el CSV.
- [ ] El agente ha descrito los datos **antes** de calcular.
- [ ] Ha detectado (solo o con tu pregunta) la lectura en otras unidades y el valor vacío.
- [ ] Los CSV originales están intactos y los scripts están guardados para repetirlos.
- [ ] El total cuadra con el informe.
- [ ] Tengo el exceso de esa semana con los dos ajustes (con todos los datos y sin la semana 3) y sé por qué difieren.
- [ ] Sé decir qué semana se sale del modelo y **qué dato lo confirma** (pista: hay una columna que no está en el informe).
- [ ] He revisado las **unidades** de lo que me dice: ¿kWh al día o en toda la semana? Los modelos a veces aciertan el número y se equivocan en la unidad.

Atención: **La lección:** un modelo dice qué es raro; no dice por qué. Que el compresor sea la causa es una hipótesis para comprobar en la planta, no una conclusión del agente.

Para llevártelo: procedimientos que ejecutan Python

Si un cálculo lo repites cada mes, guárdalo como **skill con su script**: la receta le dice al agente que **ejecute siempre el mismo programa** en vez de improvisar uno nuevo. En la página **Entorno de práctica** tienes una configuración de ejemplo con cuatro skills así; por ejemplo, `propuesta-reposicion` ejecuta `bajo_minimo.py` (como el de la parte A) y `analisis-energia-mensual` ejecuta `energia.py` (como el de la parte B). El agente redacta; el cálculo lo hace un programa que ya has revisado.

Hoja de registro

Rellénala sobre la marcha. Te sirve para el cuestionario final y para acordarte de lo que has hecho.

En las dos últimas columnas, apunta cuántos minutos calculas que te costaría hacerlo **copiando y pegando en un chat** y cuántos te ha costado **con el agente**. Fíjate sobre todo en la Práctica 5: es una tarea tuya.

Práctica	Qué pedí	Qué pasó	Qué comprobé	Min. con chat	Min. con agente
1 — Primer arranque					
2 — Provocar una aprobación					
3 — Cambiar de política					
4 — Enseñarle un procedimiento					
5 — Encargo integrador					

Práctica	Qué pedí	Qué pasó	Qué comprobé	Min. con chat	Min. con agente
6 — Python: comprobar y modelar datos					

Carril que he usado (opencode, Claude Code, Codex u otro):

Lo que me ha sorprendido:

Lo que no dejaría que hiciera sin mirar:

La primera tarea que le voy a encargar el lunes:

Recordatorio final: las cuatro reglas

1. **Copia de seguridad** de la carpeta antes de empezar.
2. **Una carpeta específica** para cada trabajo. Ni el escritorio entero ni, mucho menos, todo el disco.
3. **Nada confidencial** dentro de la carpeta de trabajo, salvo que aceptes el riesgo a conciencia.
4. **Revisa antes de aprobar y verifica al terminar.** Las dos cosas, siempre.

Hoy no has aprendido a usar un programa. Has aprendido a dirigir a alguien que trabaja muy rápido, que no sabe lo que no sabe, y que tiene acceso a tus archivos. La herramienta te va a servir; los frenos te van a proteger.